

INF2001 Introduction to Software Engineering

Milestone 2: Detailed Design, Test Plan and Wireframe

Music School Workload Allocation & Resource Scheduling System

Prepared by

Team P5-5

Student Name	Student ID
Joseph Poon	2501079
Ryan Ke	2501911
Wei Xiang	2500686
Tan Yan Qi	2500030
Zayne Lee	2503844
Lab Group:	P5 - Team 5
Date:	20 November 2026

Table of Contents

Non-Plagiarism Declaration **3**

AI Usage **4**

1 Introduction **5**

1.1 Team member contributions 5

1.2 Final requirements 5

1.3 Final Class Diagram 9

2 Component Diagram **10**

2.1 Component Architecture & Subsystems 10

2.2 Provided and Required Interface Contracts 11

2.3 Architectural Design Rationale 12

3 Detailed Design **13**

3.1 Architecture Design 13

3.2 Detailed Design (Strategy & Observer Patterns) 14

4 Testing **18**

4.1 Black Box Testing 18

4.2 White Box Testing 22

5 Wireframe **27**

5.1 Manager Scheduling & Outlier Intelligence View 27

5.2 Interactive Allocation Matrix & Conflict Feedback View 29

5.3 Teacher Availability & Timetable Inspection View 31

5.4 Audit Log Console & Governance View 33

6 References **35**

Non-Plagiarism Declaration

Students are not allowed to copy assessment solutions from another person or other places. It is the students' responsibility to guarantee that their solutions are their own work. Meanwhile, the students must also ensure that their work is not accessible by others. Where such plagiarism is detected, both of the assessments involved will receive ZERO mark.

We hereby declare that:

- We fully understand and agree to the abovementioned policy.
- We did not copy any materials from others or from other places.
- We did not share our materials with others or upload to any other places for public access.
- We agree that we will not disclose any information or material of the team project to others or upload to any other places for public access.
- We agree that our project will receive Zero mark if any misalignment with the abovementioned policies is detected.

Declared by: Group P5-5

Team members:

1. Joseph Poon (2501079)
2. Ryan Ke (2501911)
3. Wei Xiang (2500686)
4. Tan Yan Qi (2500030)
5. Zayne Lee (2503844)

Date: 20 November 2026

AI Usage

In accordance with SIT academic policy, generative AI tools were used as assistive engineering aids during Milestone 2 preparation, as summarized below:

Document Section	Generative AI Tool	Specific Prompt Intent & Usage Scope	Human Governance, Verification & Modifications
§1.2, §4.1 (Tables)	Claude Sonnet 5.5	Markdown table formatting, syntax linting, column alignment.	Replaced generic descriptions with concrete domain attributes; verified truth tables and algebraic reduction.
§1.3, §2.0, §3.2 (Diagrams)	Claude Opus 5.5	PlantUML syntax verification, styling parameter checks.	Manually authored all component boundaries, interface contracts, and GoF Strategy structures.
§2.3, §3.1 (Narrative)	Claude Sonnet 5.5	Phrasing refinement and technical editing.	Rewrote output to accurately reflect stakeholder interview evidence and statutory citations.
§4.2 (Testing Suite)	Claude Opus 5.5	Control Flow Graph edge verification and cyclomatic complexity check.	Derived basis paths manually, confirmed $V(G) = 6$, and authored executable TypeScript test cases.
§5.0 (Wireframes)	Figma + Figwright MCP	Programmatic Figma layout generation, design token binding, and high-DPI PNG asset export.	Verified requirement traceability (`FR-01` to `FR-18`), enforced WCAG 2.2 AA dual-encoded badges, and validated PDPA Section 24 PII masking.

Engineering Integrity Confirmation: All core business logic—including the 4-hour continuous teaching limit with mandatory 1-hour rest, Studio 5 acoustic drum exclusivity, monthly 19th 23:59 cutoff window, 2-stage rejection lifecycle, and multi-constraint clash validation—was derived directly from stakeholder interviews and manually verified against project specifications. Automated scheduling algorithms were rejected to preserve human manager agency.

1 Introduction

1.1 Team member contributions

Student Name	Student ID	Project Role	Authored Sections & Responsibilities
Joseph Poon	2501079	Lead Architect	Authored the UML 2.5 Component Diagram (§2.0), formulated Service Interface Contracts (§2.2), restructured the Final Class Diagram (§1.3), and designed the White-Box Testing Control Flow Graph (§4.2).
Ryan Ke	2501911	Requirements & QA Lead	Addressed M1 consultation feedback, refined master Functional Requirements (§1.2), developed Black-Box Decision Table testing methodology, and derived combinatorial reduction (§4.1).
Wei Xiang	2500686	Detailed Design Lead	Conducted architectural pattern analysis (§3.1), evaluated GoF design pattern trade-offs (§3.2.1), formalized Strategy Pattern implementation for candidate tie-breaking (§3.2.2), and authored Observer pattern specifications (§3.2.3).
Tan Yan Qi	2500030	Test & Verification Lead	Formulated interface method contracts for inter-component communication (§2.2), validated basis path coverage and cyclomatic complexity mathematics (§4.2), and synthesized white-box test suites.
Zayne Lee	2503844	Operations Lead	Developed architectural design rationale (§2.3), verified alignment between requirements, use cases, and wireframe views (§5.0), and compiled the compliance audit trail specification.

1.2 Final requirements

Following the Milestone 1 consultation, the requirement baseline was refined to eliminate scope creep, resolve vague terminology, and enforce atomic specifications meeting the 7 traits of good requirements (cohesive, complete, consistent, atomic, traceable, unambiguous, verifiable).

The bilateral peer lesson swap feature (`FR-11` in M1) was removed from the active system scope. In the stakeholder operational model, resolving lesson conflicts remains the operational responsibility of the School Manager through the job rejection workflow.

Terminology such as "workload deficit" was quantified as the remaining hours before reaching the 40.0-hour guideline (`40.0 - runningWeeklyHours`). "Teaching conditions" was clarified to mean preferred teaching days and studio assignments. To ensure sufficient scheduling options for managers while safeguarding teachers, instructors must declare at least 60.0 hours of monthly availability.

Table 1.1: Final Functional Requirements Specification

ID	Priority	Functional Requirement Specification	Updates from M1
FR-01	Must	The system shall calculate and display the running weekly and monthly allocated hours (`runningWeeklyHours`, `runningMonthlyHours`) for each teacher upon lesson confirmation.	Clarified explicit class attributes tracking running hours.
FR-02	Must	The system shall sort and display the top 3 teachers with the lowest monthly allocated hours on the manager dashboard and flag teachers whose weekly hours exceed 40.0 hours.	Removed vague term "dynamically surface"; separated UI colors into NFR.
FR-03	Must	The system shall allow managers to manually allocate unassigned 30-minute lesson slots to teachers and studios for a target week.	
FR-04	Must	The system shall display up to 3 candidate teachers side-by-side during allocation, showing current workload, preferences, and physical studio location for a specified date.	
FR-05	Should	The system shall rank candidate teachers by workload deficit (`40.0 - runningWeeklyHours`) and preferred teaching day match.	Defined workload deficit mathematically; removed vague "teaching conditions".
FR-06	Must	The system shall allow teachers to submit and edit recurring 30-minute availability slots up to 1 month ahead until 23:59 on the 19th of each month, enforcing a minimum declaration of 60.0 hours.	Added minimum 60.0-hour availability threshold.
FR-07	Must	The system shall allow teachers to specify preferred teaching days, preferred studio ID, and consecutive lesson block preferences (up to 4.0 continuous hours followed by a minimum 1.0-hour rest break) within their submitted availability window.	Clarified relationship between availability and preferences; defined consecutive clustering rules.

FR-08	Must	The system shall provide teachers with a personal weekly schedule displaying confirmed lessons, running monthly hours, and masked student contact details.	
FR-09	Must	The system shall display a mandatory consultation warning modal upon teacher job rejection and route confirmed rejection requests to the manager exception queue for re-allocation.	
FR-10	Should	The system shall flag post-19th availability changes as `Late - Pending Manager Review` and route them to the manager exception queue.	
FR-11	Should	The system shall allow IT administrators to schedule maintenance blackout windows for studios and suppress blacked-out studios from lesson allocation.	
FR-12	Should	The system shall allow managers to reassign scheduled lessons of an absent teacher to qualified and available substitute teachers.	Generalized from batch-reassignment to single-slot substitute reassignment.
FR-13	Must	The system shall allow IT administrators to provision user accounts and perform logical account deactivations for Teachers, Managers, and IT Administrators.	
FR-14	Must	The system shall manage studio configuration parameters for Studios 1 to 5 and restrict drum lesson allocations exclusively to Studio 5.	
FR-15	Must	The system shall allow managers to transition draft weekly schedules to published status and notify assigned teachers.	
FR-16	Should	The system shall calculate booked versus available studio hours across the 86 weekly operating hours and export studio utilization reports.	
FR-17	Must	The system shall track teacher weekly working hours against the 44.0-hour statutory limit under Employment Act Part IV and flag authorized overtime between 40.5 and 44.0 hours for review.	

FR-18	Must	The system shall append an immutable audit log record for every lesson allocation, managerial override, status transition, and administrative update, and allow managers to query and inspect the audit trail.	Added explicit audit trail query and inspection capability.

1.3 Final Class Diagram

The final class diagram incorporates all structural updates from Milestone 1 and Milestone 2, including explicit interface realizations, service contracts, running-hour tracking attributes, and design pattern roles.

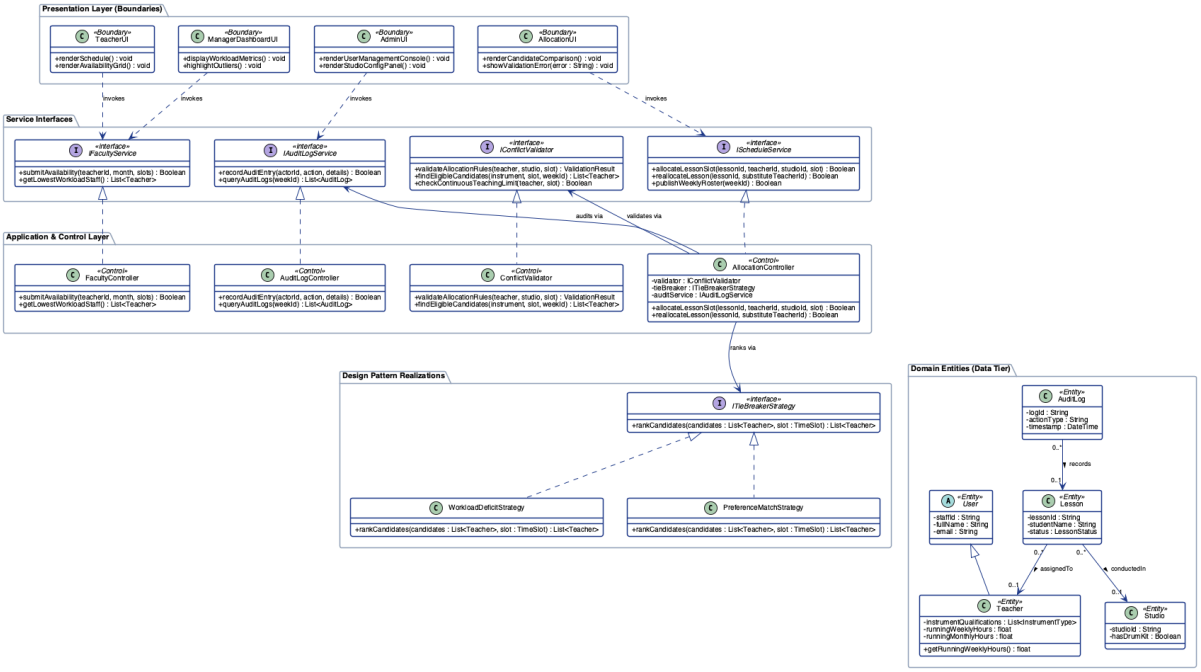


Figure 1.1: Final Class Diagram with Service Interfaces, Controllers, Strategies, and Domain Entities

2 Component Diagram

2.1 Component Architecture & Subsystems

The component diagram partitions the system into three primary internal layers enclosed within the system boundary, exposing clean interfaces and isolating external infrastructure gateways.

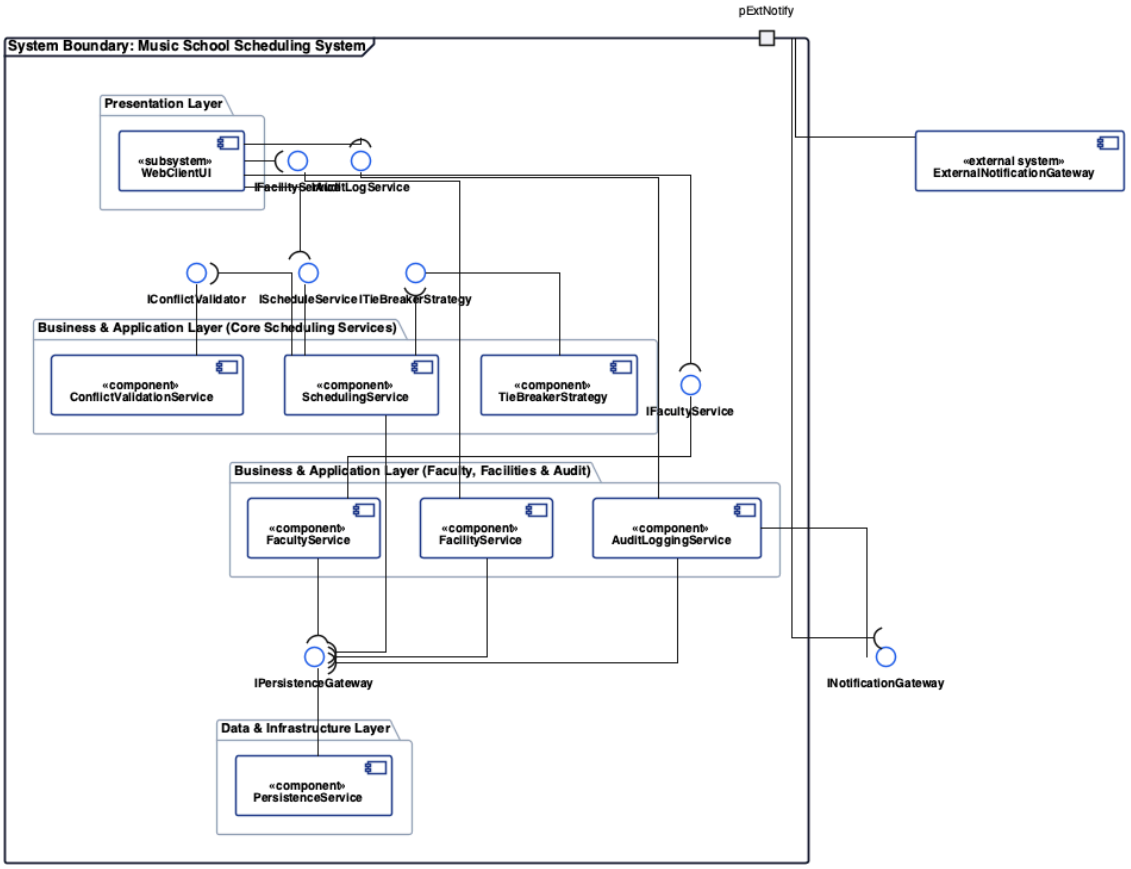


Figure 2.1: UML 2.5 Component Diagram for Music School Scheduling System

The system architecture comprises seven primary components:

1. WebClientUI: Presentation subsystem encapsulating boundary controllers (`AllocationUI`, `ManagerDashboardUI`, `TeacherUI`, `AdminUI`, `AuditLogUI`). Operates strictly as a service consumer invoking provided service interfaces.
2. SchedulingService: Central application controller orchestrating lesson slot allocation, emergency reassignment, exception resolution, and weekly roster publication.
3. ConflictValidationService: Domain validation component executing multi-constraint checks across instrument certifications, room compatibility, 4.0-hour continuous teaching limits, and statutory workload thresholds.
4. FacultyService: Manages teacher profiles, availability grids, monthly submission windows, and workload calculations.

5. FacilityService: Manages physical room metadata for Studios 1 to 5, operating schedules, instrument configurations, and servicing blackout windows.
6. AuditLoggingService: Centralized regulatory logging service maintaining append-only historical ledgers and compliance reporting tools.
7. ExternalNotificationGateway: Infrastructure adapter situated outside the system boundary, interfacing via boundary port `pExtNotify` to dispatch automated timetable alerts.

2.2 Provided and Required Interface Contracts

Every exposed interface realizes strict method contracts ensuring compile-time safety and decoupled communication across component boundaries:

`IScheduleService` (Provided by SchedulingService, Required by WebClientUI):

- `allocateLessonSlot(lessonId: String, teacherId: String, studioId: String, slot: TimeSlot): Boolean`
- `reallocateLesson(lessonId: String, substituteTeacherId: String): Boolean`
- `publishWeeklyRoster(weekId: String): Boolean`

`IConflictValidator` (Provided by ConflictValidationService, Required by SchedulingService):

- `validateAllocationRules(teacher: Teacher, studio: Studio, slot: TimeSlot): ValidationResult`
- `findEligibleCandidates(instrument: InstrumentType, slot: TimeSlot, weekId: String): List<Teacher>`
- `checkContinuousTeachingLimit(teacher: Teacher, slot: TimeSlot): Boolean`

`IFacultyService` (Provided by FacultyService, Required by WebClientUI, SchedulingService, ConflictValidationService):

- `submitAvailability(teacherId: String, month: Date, slots: List<TimeSlot>): Boolean`
- `getLowestWorkloadStaff(): List<Teacher>`
- `getTeacher(teacherId: String): Teacher`

`IFacilityService` (Provided by FacilityService, Required by WebClientUI, ConflictValidationService):

- `configureStudio(studioId: String, name: String, hasPiano: Boolean): Boolean`
- `flagMaintenanceWindow(studioId: String, start: DateTime, end: DateTime, reason: String): Boolean`
- `getStudio(studioId: String): Studio`

`IAuditLogService` (Provided by AuditLoggingService, Required by WebClientUI, SchedulingService):

- `recordAuditEntry(actorId: String, action: String, details: String): Boolean`
- `queryAuditLogs(weekId: String): List<AuditLog>`

`ITieBreakerStrategy` (Provided by TieBreakerStrategy, Required by SchedulingService):

- `rankCandidates(candidates: List<Teacher>, slot: TimeSlot): List<Teacher>`

2.3 Architectural Design Rationale

The software architecture prioritizes three critical quality attributes derived from stakeholder elicitation:

1. Modifiability: Business rules for music academies evolve frequently, including shifting statutory overtime caps, new instrument certifications, and updated break policies. Decoupling validation logic into `ConflictValidationService` and ranking algorithms into `ITieBreakerStrategy` allows developers to modify business rules without altering presentation components or data storage models.
2. Reliability & Data Integrity: Manual spreadsheet allocation resulted in undetected room double-bookings and unmonitored overtime. Centralizing placement through `IScheduleService` guarantees that every assignment passes mandatory conflict detection before persistence. Encapsulating mutation logs in `IAuditLogService` guarantees a 24-month immutable compliance history satisfying Ministry of Manpower mandates.
3. Usability & Operational Responsiveness: School managers construct complex weekly schedules under strict deadlines. Decomposing the system into granular service interfaces enables sub-300ms conflict checks on candidate selection, giving managers immediate visual feedback without reloading full page states.

3 Detailed Design

3.1 Architecture Design

The system implements a strict Hierarchical Layered Architecture organized into three closed tiers: Presentation Layer, Business and Application Layer, and Data and Infrastructure Layer.

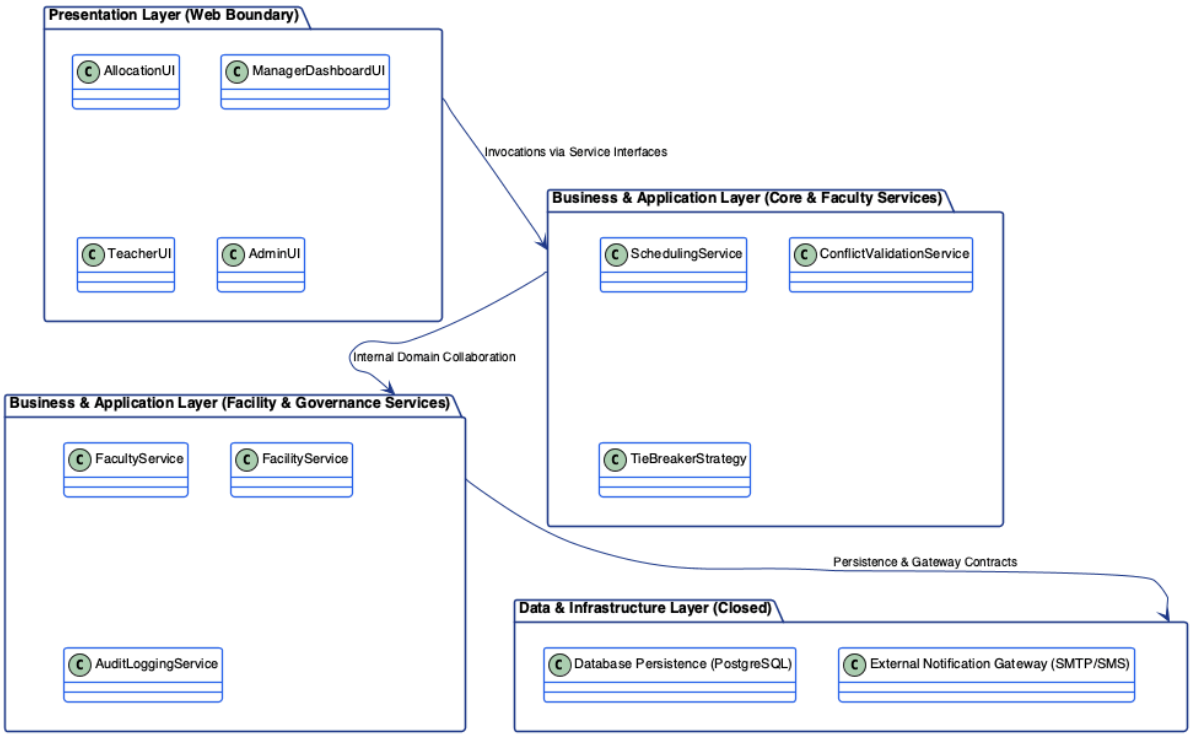


Figure 3.1: 3-Tier Hierarchical Layered Architecture for Music School Scheduling System

In accordance with closed layering rules, layer-skipping is strictly prohibited. Requests flow sequentially downward through the business controllers, and domain data returns synchronously back through the calling hierarchy. Architectural Pattern Comparison:

Evaluated Architecture	Structural Characteristics	Suitability Analysis for Scheduling System
Hierarchical Layered (Selected)	Partitions system horizontally into closed Presentation, Business, and Persistence layers. Each layer depends only on the layer immediately below it.	Selected: Enforces clean separation of concerns, guarantees that business rules and statutory overtime checks cannot be bypassed by UI components, and simplifies unit test isolation.

Model-View-Controller (MVC)	Separates UI rendering (View) and user inputs (Controller) from domain entities (Model).	Adopted within Presentation Tier: Boundary views capture user inputs and delegate to controllers, but MVC alone lacks formal boundaries for complex domain validation services.
Microservices Architecture	Distributes application logic across independent, network-partitioned service processes.	Rejected: Introduces unnecessary network latency, distributed transaction complexity, and DevOps overhead for an academy with 9 teachers and 5 studios.

3.2 Detailed Design

3.2.1 Design Pattern Evaluation & Trade-off Analysis

To resolve detailed design problems within the scheduling core, four Gang of Four (GoF) patterns were evaluated against operational requirements:

Evaluated Pattern	Target Operational Problem	Architectural Trade-offs & Analysis	Selection Outcome
Strategy Pattern	Dynamic candidate teacher ranking across competing operational heuristics (workload deficit vs preference match).	Pros: Decouples candidate sorting algorithms from allocation controllers; satisfies Open/Closed Principle. Cons: Increases class count slightly. Verdict: Long-term extensibility outweighs minimal class boilerplate.	Selected (Implemented)
Observer Pattern	Decoupling schedule publication events from automated email, SMS, and dashboard notification dispatch.	Pros: Broadcasts schedule publication events to multiple communication channels without coupling scheduling controllers to notification protocols. Cons: Unsubscribing listeners requires explicit lifecycle management to avoid memory retention.	Selected (Implemented)

Factory Method	Instantiating specialized User accounts (Teacher, Manager, ITAdministrator) during onboarding.	Pros: Encapsulates role creation. Cons: Account onboarding occurs infrequently; adds unnecessary abstraction over simple constructors.	Evaluated (Alternative)
Singleton Pattern	Providing global access to the scheduling engine or system configuration registry.	Pros: Single point of access. Cons: Introduces global state, tightly couples callers to concrete implementations, and hinders test isolation.	Evaluated (Rejected)

3.2.2 Strategy Pattern (Selected)

Operational Problem: When allocating an unassigned lesson, multiple teachers often satisfy all hard constraints (qualified, available, within continuous teaching limits, under 44.0 hours). The system must rank these candidates to guide manager selection. Under nominal operations, management prioritizes faculty equity by ranking teachers by workload deficit (`40.0 - runningWeeklyHours`). In specialized cases, management prioritizes teacher satisfaction by ranking candidates by preferred teaching days and studio clustering.

Hardcoding this ranking logic into `AllocationController` violates the Open/Closed Principle (OCP) and Single Responsibility Principle (SRP). Adding new ranking heuristics requires modifying and recompiling core scheduling classes. The Strategy Pattern decouples algorithmic ranking from scheduling orchestration.

Structural Template:

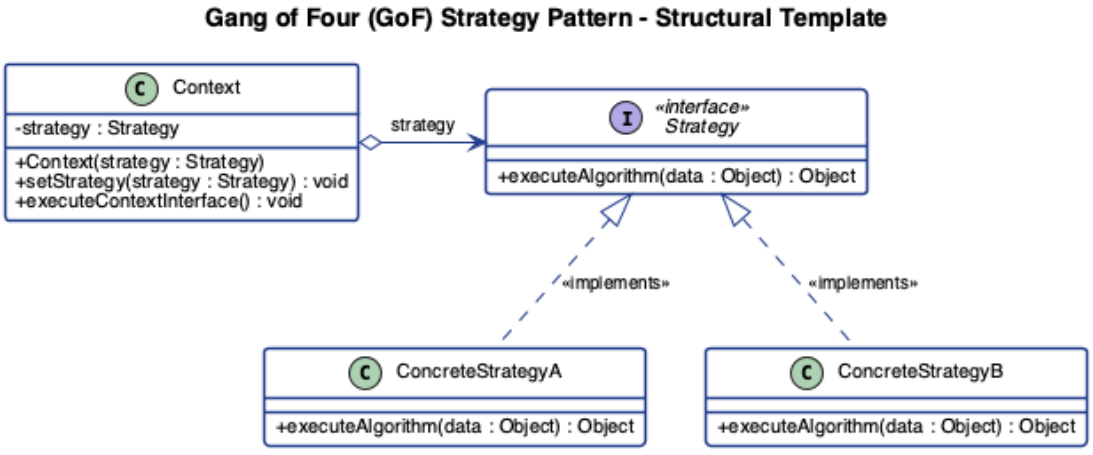


Figure 3.2: Gang of Four Strategy Pattern Standard Structural Template

Before Pattern Implementation:

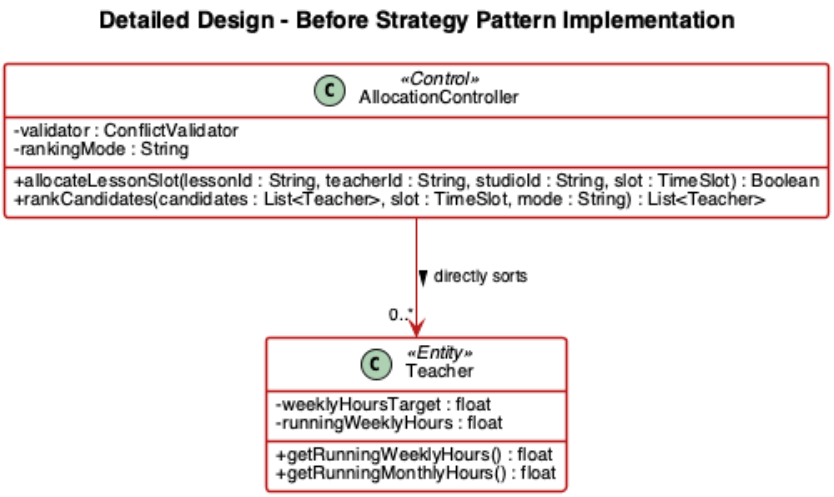


Figure 3.3: Class Excerpt Before Strategy Implementation (Monolithic Conditional Sorting)

After Strategy Pattern Implementation:

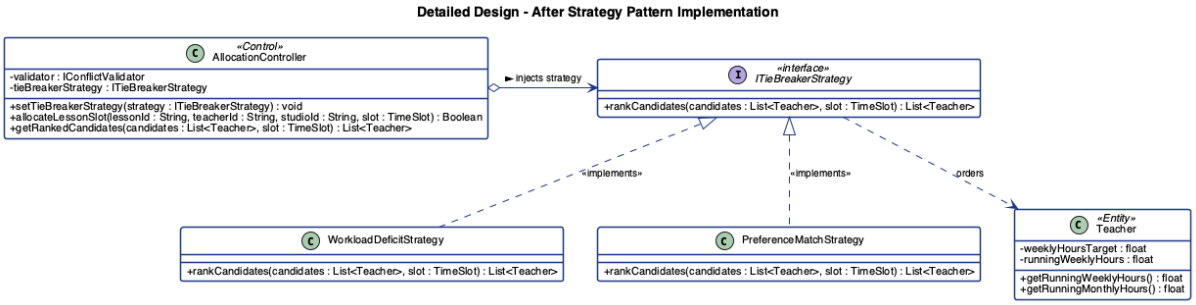


Figure 3.4: Class Excerpt After Strategy Implementation (Extensible Policy Decoupling)

In this refactored structure, `AllocationController` acts as the Context, holding a reference to the `ITieBreakerStrategy` interface. Concrete strategies `WorkloadDeficitStrategy` and `PreferenceMatchStrategy` encapsulate independent sorting algorithms. When managers toggle ranking preferences in `AllocationUI`, the controller dynamically swaps the active strategy instance without modifying orchestration code.

3.2.3 Observer Pattern (Selected)

Operational Problem: When a manager completes and publishes a weekly schedule (`UC-05`), multiple decoupled subsystems require notification: assigned teachers must receive mobile roster alerts, administrators require audit event records, and dashboard metrics must synchronize. Direct calls from `AllocationController` to external mail or SMS APIs tightly couple core scheduling logic to communication protocols.

Applying the Observer Pattern establishes a one-to-many dependency, allowing `AllocationController` (Subject) to broadcast state transitions to registered listeners (Observers) without coupling to concrete notification implementations.

Structural Template:

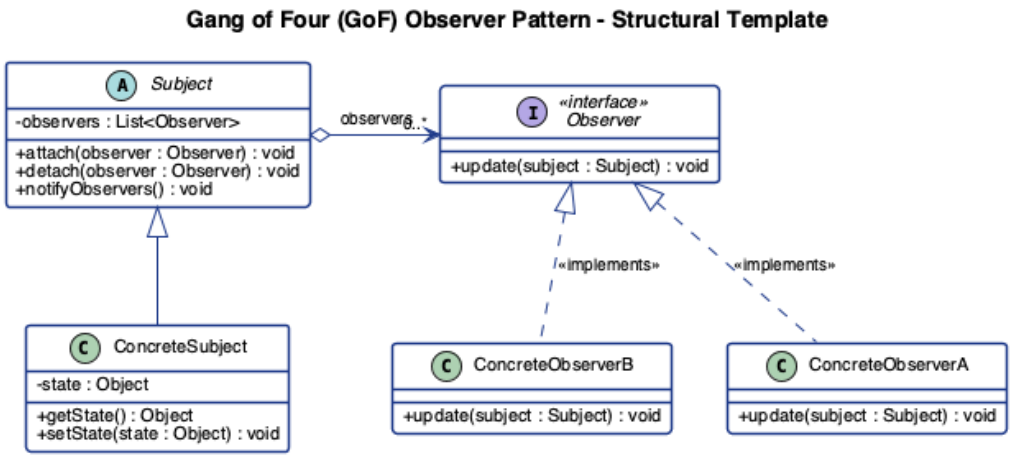


Figure 3.5: Gang of Four Observer Pattern Standard Structural Template

After Observer Pattern Implementation:

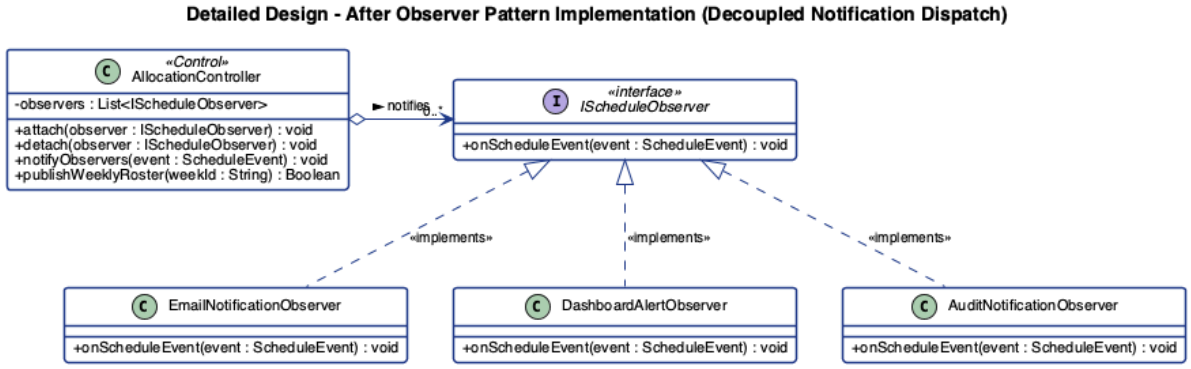


Figure 3.6: Class Excerpt After Observer Implementation (Decoupled Notification Dispatch)

In this structure, `AllocationController` maintains an internal subscriber list through the `IScheduleObserver` interface. When `publishWeeklyRoster()` commits, it iterates through observers invoking `onScheduleEvent()`. Concrete subscribers `EmailNotificationObserver`, `DashboardAlertObserver`, and `AuditNotificationObserver` handle delivery independently.

4 Testing

4.1 Black Box Testing

Black-box testing targets the flagship operational method `allocateLessonSlot()` in `SchedulingService`. This method evaluates multi-constraint feasibility before confirming a 30-minute lesson placement.

4.1.1 Cause-Effect Specification

The allocation decision evaluates seven core conditions:

- Causes (Inputs and Pre-conditions):
    - C1 {qual}: Teacher holds certified instrument qualification for lesson (`True` / `False`)
    - C2 {avail}: Teacher declared available for selected 30-minute timeslot (`True` / `False`)
    - C3 {fatigue}: Teacher continuous teaching duration ≤ 4.0 hours if assigned (`True` / `False`)
    - C4 {cap}: Teacher projected weekly workload ≤ 44.0 statutory ceiling (`True` / `False`)
    - C5 {ot}: Teacher projected weekly workload ≤ 40.0 internal guideline (`True` / `False`)
    - C6 {room}: Selected studio is unoccupied and not under maintenance blackout (`True` / `False`)
    - C7 {drum}: Lesson instrument is Drums and selected room is Studio 5 (`True` / `False`)
- Effects (System Actions and Outputs):
    - E1 {nominal}: Lesson confirmed nominally; status set to `Confirmed`; running workload incremented by 0.5h.
    - E2 {advisory}: Soft-warning overtime prompt triggered; manager override required before confirmation.
    - E3 {qual_err}: Allocation rejected: "Teacher not qualified for instrument".
    - E4 {avail_err}: Allocation rejected: "Teacher unavailable for selected timeslot".
    - E5 {fatigue_err}: Allocation rejected: "Continuous teaching limit breached (mandatory 1.0h rest required)".
    - E6 {stat_err}: Allocation rejected: "Statutory 44.0h weekly cap breached (Employment Act Part IV)".
    - E7 {room_err}: Allocation rejected: "Studio occupied or under maintenance blackout".
    - E8 {drum_err}: Allocation rejected: "Drum lessons restricted exclusively to Studio 5".

4.1.2 Complete Decision Table (Combination Space)

With 7 binary conditions, the complete unreduced combination space encompasses $2^7 = 128$ distinct rules. Because the allocation engine evaluates constraints hierarchically, an invalid condition at an earlier stage short-circuits subsequent checks.

Table 4.1: Complete Decision Table Space Summary (128 Rules)

Rule Group	Condition States (C1, C2, C3, C4, C5, C6, C7)	Rule Count	Resulting System Effect
Group 1 (Unqualified)	C1 = False, C2..C7 in {True, False}	64	E3 {qual_err} (Qualification error halts allocation)
Group 2 (Unavailable)	C1 = True, C2 = False, C3..C7 in {True, False}	32	E4 {avail_err} (Availability conflict halts allocation)
Group 3 (Fatigue Breach)	C1 = T, C2 = T, C3 = F, C4..C7 in {True, False}	16	E5 {fatigue_err} (Continuous teaching limit breached)
Group 4 (Statutory Breach)	C1 = T, C2 = T, C3 = T, C4 = F, C5..C7 in {True, False}	8	E6 {stat_err} (Statutory 44.0h legal cap breached)
Group 5 (Room Conflict)	C1 = T, C2 = T, C3 = T, C4 = T, C5 in {T,F}, C6 = F, C7 in {T,F}	4	E7 {room_err} (Studio occupied or in servicing)
Group 6 (Drum Exclusivity)	C1 = T, C2 = T, C3 = T, C4 = T, C5 in {T,F}, C6 = T, C7 = F	2	E8 {drum_err} (Drums not in Studio 5)
Group 7 (Overtime Advisory)	C1 = T, C2 = T, C3 = T, C4 = T, C5 = F, C6 = T, C7 = T	1	E2 {advisory} (Advisory overtime 40.5h–44.0h prompt)
Group 8 (Nominal Success)	C1 = T, C2 = T, C3 = T, C4 = T, C5 = T, C6 = T, C7 = T	1	E1 {nominal} (Allocation confirmed nominally)
Total Combinations		128	

4.1.3 Reduced Decision Table

By collapsing don't-care conditions (`-`) where upstream validation failures supersede downstream checks, the 128 combinations reduce systematically to 8 canonical test rules. Rule 8 is partitioned into two boundary tests covering nominal hours vs advisory overtime, yielding 9 actionable test cases.

Table 4.2: Reduced Decision Table

Conditions / Rules	R1	R2	R3	R4	R5	R6	R7	R8	R9
C1 {qual}	**F**	**T**	**T**	**T**	**T**	**T**	**T**	**T**	**T**
C2 {avail}	-	**F**	**T**	**T**	**T**	**T**	**T**	**T**	**T**
C3 {fatigue}	-	-	**F**	**T**	**T**	**T**	**T**	**T**	**T**
C4 {cap}	-	-	-	**F**	**T**	**T**	**T**	**T**	**T**
C5 {ot}	-	-	-	-	**T**	**F**	**T**	**T**	**F**
C6 {room}	-	-	-	-	**F**	**F**	**T**	**T**	**T**
C7 {drum}	-	-	-	-	-	-	**F**	**T**	**T**
Effects									
E1 {nominal}								**X**	
E2 {advisory}									**X**
E3 {qual_err}	**X**								
E4 {avail_err}		**X**							
E5 {fatigue_err}			**X**						
E6 {stat_err}				**X**					
E7 {room_err}					**X**	**X**			
E8 {drum_err}							**X**		

4.1.4 Derived Black-Box Test Cases

Boundary Value Analysis (BVA) was applied to quantitative boundaries: 4.0h continuous teaching, 40.0h overtime threshold, and 44.0h statutory ceiling.

Table 4.3: Derived Test Cases for allocateLessonSlot()

Test ID	Trace Rule	Test Scenario & Input Parameters	Expected System Outcome
TC-BB-01	R1	Teacher uncertified for Drums assigned to Drum lesson.	Rejected with `E3 {qual_err}`.

TC-BB-02	R2	Teacher declared unavailable on Tuesday 14:00 assigned to 14:00 slot.	Rejected with `E4 {avail_err}`.
TC-BB-03	R3	Teacher has taught 4.0h continuously; assigned consecutive 0.5h slot without break.	Rejected with `E5 {fatigue_err}`.
TC-BB-04	R4	Teacher currently at 44.0h weekly workload; assigned additional 0.5h slot.	Rejected with `E6 {stat_err}`.
TC-BB-05	R5	Studio 2 occupied by another student during target 15:00 slot.	Rejected with `E7 {room_err}`.
TC-BB-06	R6	Studio 3 scheduled under piano tuning blackout window.	Rejected with `E7 {room_err}`.
TC-BB-07	R7	Drum lesson placed into Studio 1 (Piano-only studio).	Rejected with `E8 {drum_err}`.
TC-BB-08	R8	Certified teacher, available, 2.0h continuous block, weekly hours 32.0h assigned Studio 5 drum slot.	Confirmed with `E1 {nominal}`; workload becomes 32.5h.
TC-BB-09	R9	Certified teacher, available, fresh block, weekly hours 40.0h assigned 0.5h slot (reaches 40.5h).	Prompts `E2 {advisory}`; confirms upon manager override; workload becomes 40.5h.

4.2 White Box Testing

White-box testing targets `ConflictValidator.findEligibleCandidates()`. This method filters the active faculty pool to identify instructors eligible to take a specific student lesson slot.

4.2.1 Algorithm Pseudocode

```
function findEligibleCandidates(instrument: InstrumentType,
slot: TimeSlot, weekId: String): List<Teacher> {
    let eligibleCandidates: List<Teacher> = new ArrayList();
    let facultyPool: List<Teacher> =
facultyService.getActiveFaculty();
    let i: number = 0;
    while (i < facultyPool.size()) {
        let candidate: Teacher = facultyPool.get(i);
        if (candidate.hasInstrumentQualification(instrument)) {
            if (candidate.isAvailable(slot)) {
                if (this.checkContinuousTeachingLimit(candidate,
slot)) {
                    if (candidate.getRunningWeeklyHours() + 0.5
<= 44.0) {
                        eligibleCandidates.add(candidate);
                    }
                }
            }
        }
        i = i + 1;
    }
    return eligibleCandidates;
}
```

4.2.2 Control Flow Graph (CFG)

The CFG represents statement execution blocks as nodes and branch control transfers as directed edges:

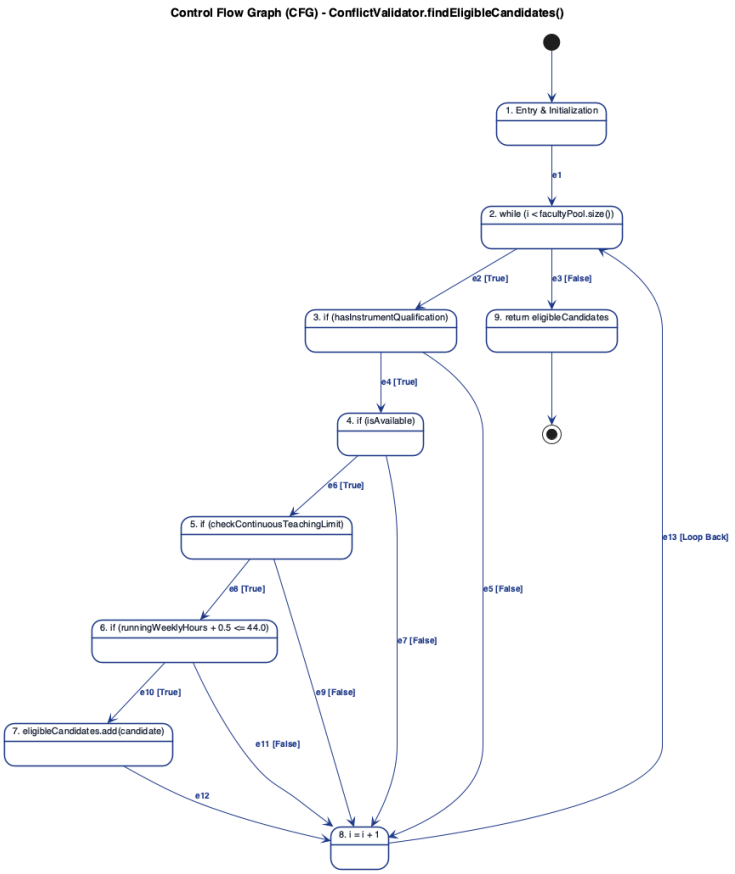


Figure 4.1: Control Flow Graph for ConflictValidator.findEligibleCandidates()

4.2.3 Cyclomatic Complexity Calculation

Cyclomatic complexity V(G) is calculated strictly using the canonical McCabe formula:

V(G) = E - N + 2P

- Number of edges (E): 13 directed edges (e1 to e13)
- Number of nodes (N): 9 execution nodes (N1 to N9)
- Connected components (P): 1 single program flow

V(G) = 13 - 9 + 2(1) = 6

A cyclomatic complexity of 6 dictates that the method contains exactly 6 linearly independent execution paths required to achieve 100% statement and 100% branch coverage simultaneously.

4.2.4 Linearly Independent Basis Paths

1. Path 1: 1 -> 2 -> 9 (Empty faculty list; loop condition immediately false)
2. Path 2: 1 -> 2 -> 3 -> 8 -> 2 -> 9 (Candidate fails instrument qualification check)
3. Path 3: 1 -> 2 -> 3 -> 4 -> 8 -> 2 -> 9 (Candidate qualified, but fails timeslot availability check)
4. Path 4: 1 -> 2 -> 3 -> 4 -> 5 -> 8 -> 2 -> 9 (Candidate qualified and available, but breaches 4.0h continuous teaching limit)

5. Path 5: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 8 -> 2 -> 9 (Candidate passes fatigue check, but projected hours breach 44.0h statutory ceiling)
6. Path 6: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 2 -> 9 (Candidate satisfies all constraints; appended to eligible pool)

Table 4.4: Basis Path Test Cases for findEligibleCandidates()

Test ID	Path Traversed	Input Candidate State & Conditions	Expected Execution Outcome
TC-WB-01	Path 1	`facultyPool.size() = 0`	Loop bypassed; returns empty list immediately.
TC-WB-02	Path 2	Candidate possesses Piano certification; queried for Drums.	Node 3 evaluates False; branch jumps to Node 8; candidate excluded.
TC-WB-03	Path 3	Candidate certified for Drums; slot is Tuesday 14:00; availability is False.	Node 4 evaluates False; branch jumps to Node 8; candidate excluded.
TC-WB-04	Path 4	Candidate qualified and available; already taught 4.0h continuous block.	Node 5 evaluates False; branch jumps to Node 8; candidate excluded.
TC-WB-05	Path 5	Candidate qualified, available, fresh block; `runningWeeklyHours = 43.8h`.	Node 6 evaluates False ($43.8 + 0.5 = 44.3 > 44.0$); candidate excluded.
TC-WB-06	Path 6	Candidate qualified, available, fresh block; `runningWeeklyHours = 28.0h`.	All nodes evaluate True; Node 7 adds candidate; returns list of size 1.

4.2.5 Executable Unit Test Suite Implementation

```
import { describe, it, expect, beforeEach } from 'vitest';
import { ConflictValidator } from
'../src/services/ConflictValidator';
import { InstrumentType, TimeSlot, Teacher } from
'../src/domain/entities';

describe('ConflictValidator.findEligibleCandidates() - White Box
Basis Paths', () => {
    let validator: ConflictValidator;
    let mockSlot: TimeSlot;

    beforeEach(() => {
        validator = new ConflictValidator();
        mockSlot = new TimeSlot('2026-11-17T14:00:00', '2026-11-
17T14:30:00');
    });

    it('TC-WB-01 (Path 1): should handle empty faculty pool', ()
=> {
        const result =
validator.findEligibleCandidates(InstrumentType.PIANO, mockSlot,
[]);
        expect(result).toHaveLength(0);
    });

    it('TC-WB-02 (Path 2): should exclude candidates lacking
instrument qualification', () => {
        const candidate = new Teacher('T-01', 'Elena Lim',
[InstrumentType.VIOLIN], 20.0, true);
        const result =
validator.findEligibleCandidates(InstrumentType.PIANO, mockSlot,
[candidate]);
        expect(result).toHaveLength(0);
    });

    it('TC-WB-03 (Path 3): should exclude qualified candidates
who are unavailable', () => {
        const candidate = new Teacher('T-02', 'Sarah Jenkins',
[InstrumentType.PIANO], 20.0, false);
        const result =
validator.findEligibleCandidates(InstrumentType.PIANO, mockSlot,
[candidate]);
        expect(result).toHaveLength(0);
    });

    it('TC-WB-04 (Path 4): should exclude candidates breaching
```

```
4.0h continuous limit', () => {
        const candidate = new Teacher('T-03', 'David Koh',
[InstrumentType.PIANO], 20.0, true);
        candidate.setContinuousTeachingHours(4.0);
        const result =
validator.findEligibleCandidates(InstrumentType.PIANO, mockSlot,
[candidate]);
        expect(result).toHaveLength(0);
    });

    it('TC-WB-05 (Path 5): should exclude candidates breaching
44.0h statutory ceiling', () => {
        const candidate = new Teacher('T-04', 'Chloe Teo',
[InstrumentType.PIANO], 43.8, true);
        candidate.setContinuousTeachingHours(1.0);
        const result =
validator.findEligibleCandidates(InstrumentType.PIANO, mockSlot,
[candidate]);
        expect(result).toHaveLength(0);
    });

    it('TC-WB-06 (Path 6): should include candidates satisfying
all operational constraints', () => {
        const candidate = new Teacher('T-05', 'Marcus Tan',
[InstrumentType.PIANO], 28.0, true);
        candidate.setContinuousTeachingHours(1.5);
        const result =
validator.findEligibleCandidates(InstrumentType.PIANO, mockSlot,
[candidate]);
        expect(result).toHaveLength(1);
        expect(result[0].getStaffId()).toBe('T-05');
    });
});
```

5 Wireframe

The user interface was engineered to bridge functional requirements with human-centered ergonomics, ensuring frictionless decision-support across all 17 finalized use cases. Wireframes were designed using high-fidelity Figma components and exported via automated browser tooling.

5.1 Manager Scheduling & Outlier Intelligence View

- Linked Use Case: `UC-01` (View Workload Dashboard & Outliers)
- Supported Requirements: `FR-01`, `FR-02`, `FR-03`, `NFR-USE-02`.
- Screen Purpose: Serves as the operational landing cockpit for school managers, summarizing real-time faculty capacity across active instructors and proactively highlighting staffing inequities prior to weekly roster creation.
- Key User Interactions:
    1. Workload Progress Indicators: Renders real-time capacity meters color-coded according to `NFR-USE-02` (Green < 35.0h, Amber 35.0h to 40.0h, Red > 40.0h).
    2. Top 3 Under-Allocated Outliers: Automatically bubbles up the 3 instructors with the lowest allocated hours, enabling managers to assign pending lessons with one click.
    3. Statutory Ceiling Warning Badges: Highlights any instructor approaching or reaching the statutory 44.0-hour maximum with a red lock badge, blocking additional assignments.

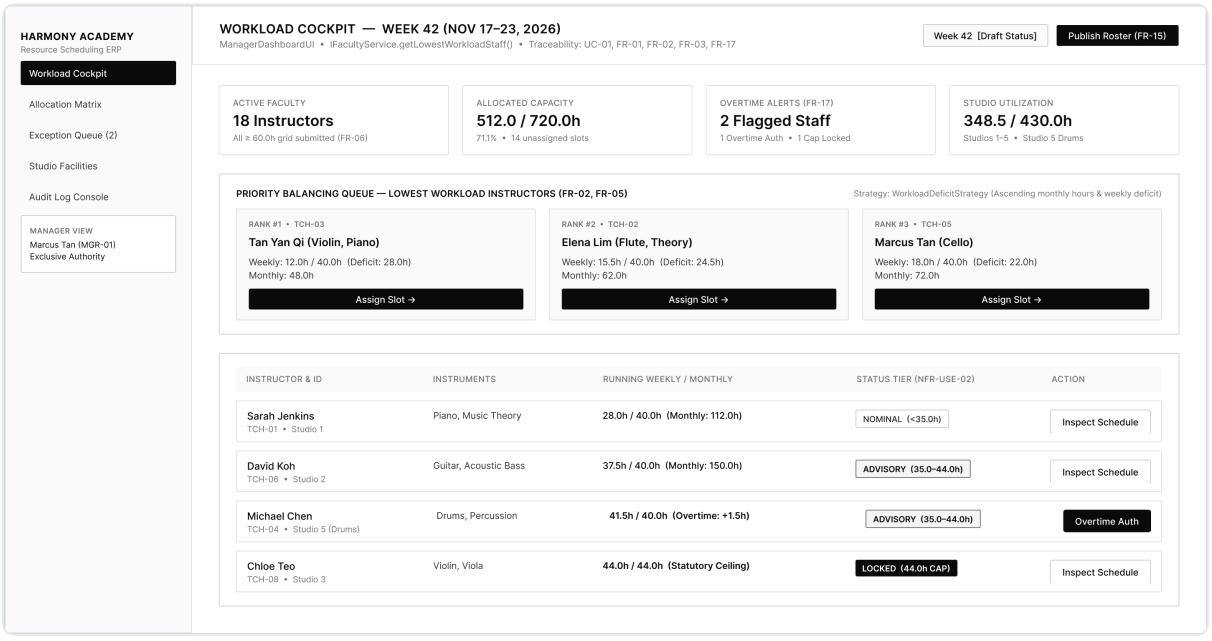


Figure 5.1: Manager Scheduling & Outlier Intelligence Cockpit Wireframe (Figma — UC-01, FR-01, FR-02, FR-03, FR-15, FR-17)

5.2 Interactive Allocation Matrix & Conflict Feedback View

- Linked Use Case: `UC-02` (Allocate Lesson Slot), `UC-03` (Review & Re-allocate), `UC-06` (Emergency Substitute).
- Supported Requirements: `FR-04`, `FR-05`, `FR-06`, `FR-16`, `FR-18`.
- Screen Purpose: Provides managers with side-by-side decision support when placing a 30-minute student lesson slot, executing sub-300ms conflict checks against qualifications, availability, 4-hour fatigue limits, and room compatibility.
- Key User Interactions:
    1. Candidate Comparison Panel: Presents up to 3 pre-filtered candidates side-by-side with running weekly hours, monthly totals, preferences, and studio location for that date.
    2. Automated Tie-Breaking Strategy: Injects `ITieBreakerStrategy` (Workload Deficit or Preference Match), surfacing the most eligible candidate with a "Recommended" badge.
    3. Studio Isolation Enforcement: Restricts studio pickers dynamically (Studio 5 locked for Drums; maintenance windows disabled).
    4. Advisory Overtime Modal: If assigning the lesson pushes the instructor between 40.5h and 44.0h, triggers an advisory confirmation modal requiring mandatory justification before commit.

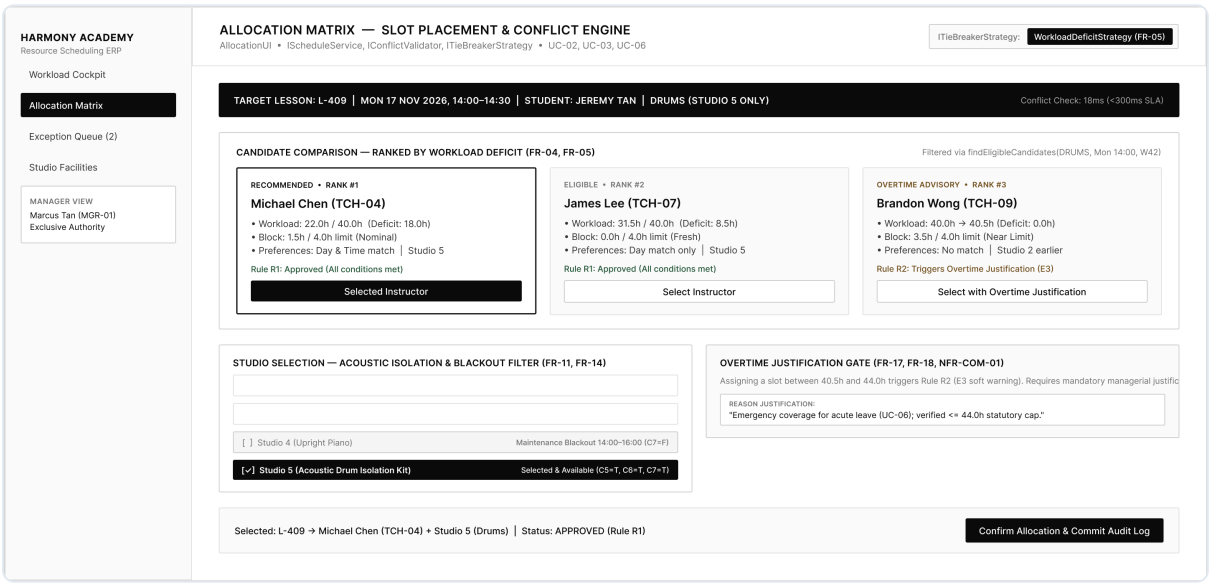


Figure 5.2: Interactive Allocation Matrix & Multi-Constraint Conflict Feedback Wireframe (Figma — UC-02, UC-03, UC-06, FR-04, FR-05, FR-14)

5.3 Teacher Availability & Timetable Inspection View

- Linked Use Case: UC-07 (Submit Availability), UC-08 (Submit Preferences), UC-09 (View Schedule), UC-10 (Rejection).
- Supported Requirements: FR-07, FR-08, FR-09, FR-10, FR-11, NFR-SEC-01.
- Screen Purpose: Provides instructors with an ergonomic, responsive portal to declare recurring monthly availability, submit consecutive clustering preferences, inspect published timetables, and initiate rejection tickets if necessary.
- Key User Interactions:
    1. Availability Grid (30-min increments): Visual click-and-drag grid; enforces the minimum 60.0-hour declaration rule before allowing submission.
    2. 19th 23:59 Cutoff Clock: Displays a real-time countdown timer. Edits prior to cutoff save automatically; edits after cutoff require manager review (UC-04).
    3. Masked Student Contact PII: Displays student first names and instrument levels, strictly masking contact details in compliance with Singapore PDPA Section 24.
    4. Job Rejection Modal: Displays a consultation advisory modal, captures mandatory reason codes, and forwards rejected slots to the manager exception queue (UC-03).

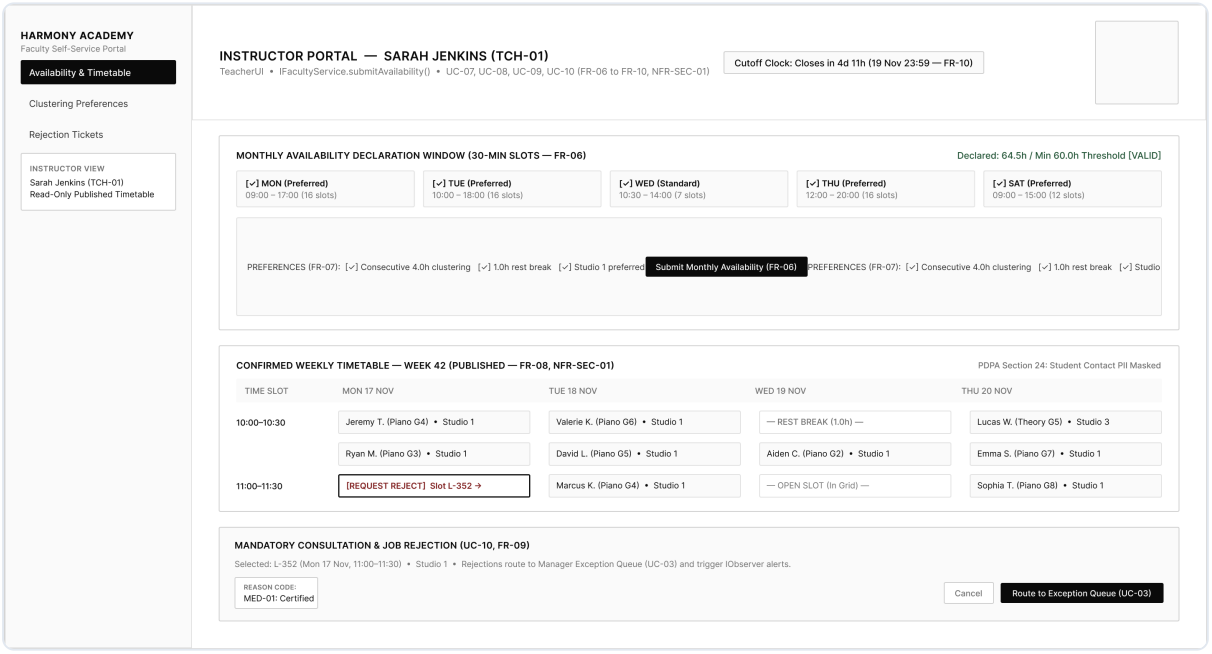


Figure 5.3: Teacher Availability Declaration, PDPA-Masked Timetable & Rejection Portal Wireframe (Figma — UC-07 to UC-10, FR-06 to FR-10)

5.4 Audit Log Console & Governance View

- Linked Use Case: `UC-11` (Inspect System Audit Log), `UC-17` (Review Labor Compliance).
- Supported Requirements: `FR-18`, `NFR-COM-01`, `NFR-COM-02`.
- Screen Purpose: Furnishes school administrators and compliance officers with an unalterable chronological ledger of all scheduling mutations, overtime overrides, and system access changes.
- Key User Interactions:
    1. Multi-Parameter Filtering: Enables instant querying by date range, staff ID, action type (`ALLOCATION_COMMIT`, `OVERTIME_OVERRIDE`, `REJECTION_RESOLVED`), or entity ID.
    2. Cryptographic Integrity Badges: Validates and displays SHA-256 tamper-evident integrity hashes for each record.
    3. Export Compliance Bundle: Generates exportable CSV/PDF audit reports satisfying Singapore Ministry of Manpower 24-month record retention mandates.

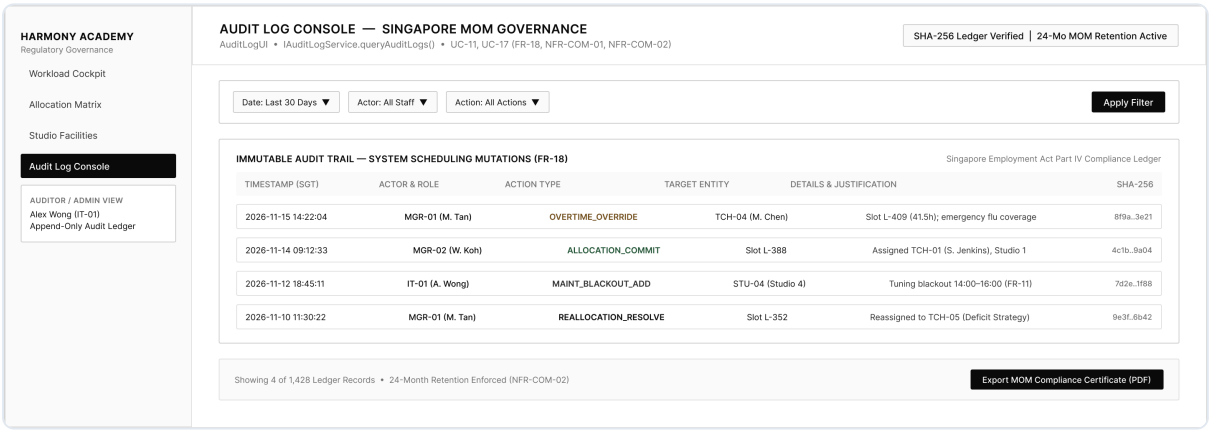


Figure 5.4: Regulatory Compliance & Immutable SHA-256 Audit Trail Console Wireframe (Figma — UC-11, UC-17, FR-18, NFR-COM-01, NFR-COM-02)

6 References

1. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley Professional.
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
3. McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
4. Ministry of Manpower Singapore. (2026). *Employment Act (Chapter 91) — Part IV: Hours of Work, Rest Days and Other Conditions of Service*. Singapore Statutes Online.
5. Personal Data Protection Commission Singapore. (2024). *Advisory Guidelines on Key Concepts in the Personal Data Protection Act (PDPA Section 24: Protection Obligation)*.
6. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
7. World Wide Web Consortium (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2 — Rule 1.4.1: Use of Color*. W3C Recommendation.